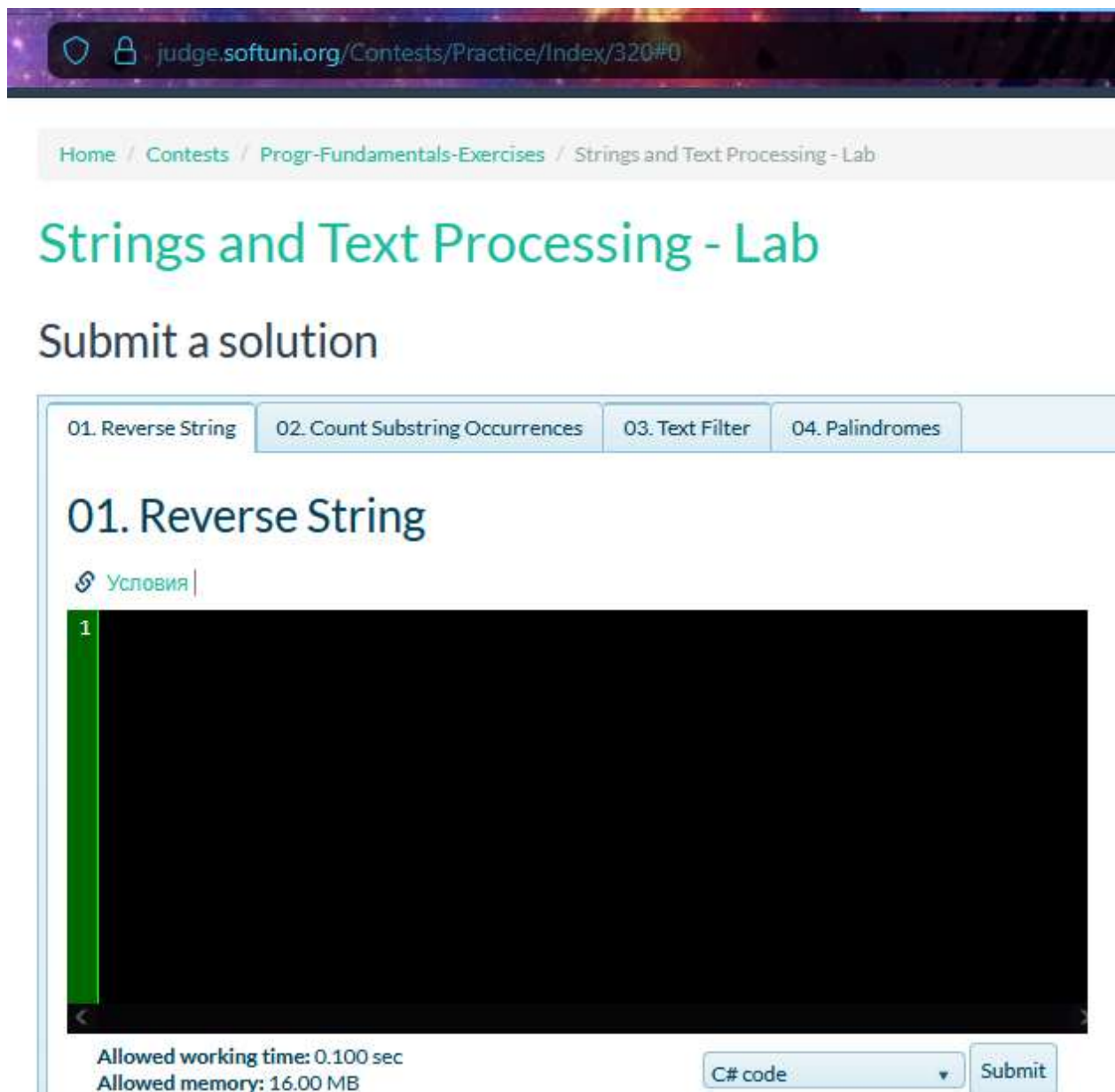


Клас String Builder – преговор с допълнение

SoftUni – Judge – Безплатен по книгата <https://judge.softuni.org/Contests/Practice/Index/320#0>



Home / Contests / Progr-Fundamentals-Exercises / Strings and Text Processing - Lab

Strings and Text Processing - Lab

Submit a solution

01. Reverse String 02. Count Substring Occurrences 03. Text Filter 04. Palindromes

01. Reverse String

Условия

1

Allowed working time: 0.100 sec
Allowed memory: 16.00 MB

C# code Submit

Най-често сме сравнявали два низа чрез `==` или `!=`

`.Compare()` – сравнява 2 низа **лексикографски** (по азбучен ред), като резултатът е `int`:

0 – двата са с равна дължина

> 0 – вторият е по-дълъг

< 0 – вторият е по-къс

.Compare() + bool ignoreCase – за case sensitive – дали да игнорира/различава главни от малки букви

```
string shortText = "Сняг";
string longText = "Все още е зима";
string shortTextSmallCase = "сняг";

Console.WriteLine(string.Compare(shortText, longText)); // 1
Console.WriteLine(string.Compare(shortText, shortTextSmallCase, false)); // 1
Console.WriteLine(string.Compare(shortText, shortTextSmallCase, true)); // 0
```

int string.Compare(string? strA, string? strB, bool ignoreCase) (+ 9 overloads)
Compares two specified `string` objects, ignoring or honoring their case, and returns an integer that indicates their relative position in the sort order.

Returns:
A 32-bit signed integer that indicates the lexical relationship between the two comparands.

Value – Condition

- Less than zero** – `strA` precedes `strB` in the sort order.
- Zero** – `strA` occurs in the same position as `strB` in the sort order.
- Greater than zero** – `strA` follows `strB` in the sort order.

Разгледахме съединяване на низове чрез **+**, но може да се използва и **+=**.
Знакът **+** е бавен.

.Concat() – значително по-бърз

```
Console.WriteLine(string.Concat());
```

string string.Concat(string? str0, string? str1, string? str2)
Concatenates three specified instances of `string`.
str0: The first string to concatenate.

```
Console.WriteLine(string.Concat(longText, " и вали ", shortText.ToLower()));
```

Все още е зима и вали сняг

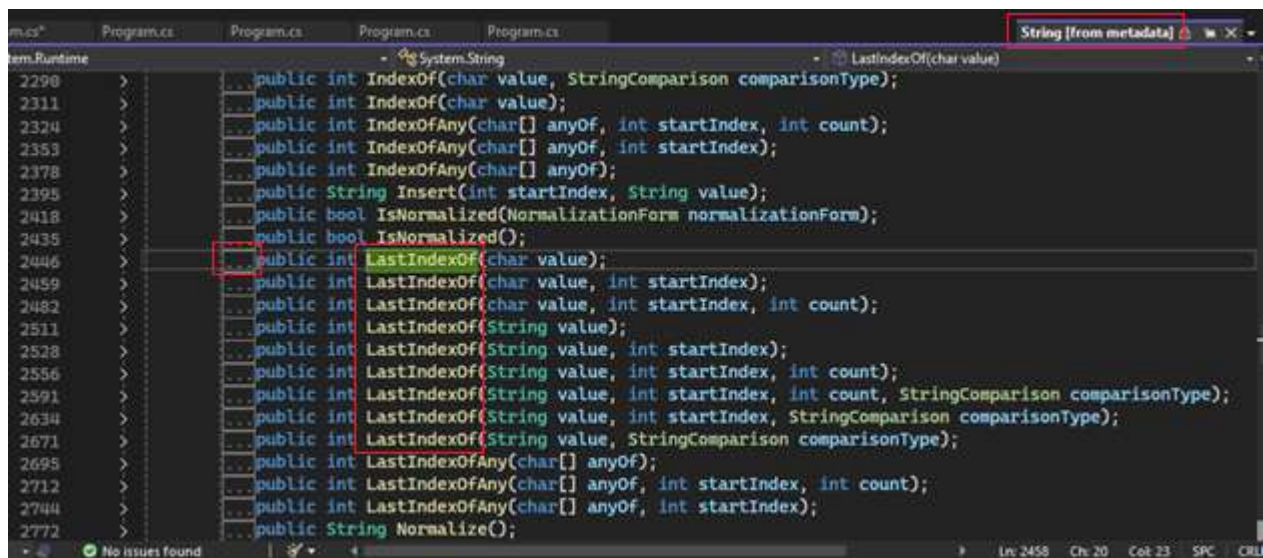
```
Console.WriteLine(string.LastIndexOf());
```

int string.LastIndexOf(char value) (+ 8 overloads)
Reports the zero-based index position of the last occurrence of a specified Unicode character within this instance.

Returns:
The zero-based index position of `value` if that character is found, or **-1 if it is not**. **negative**

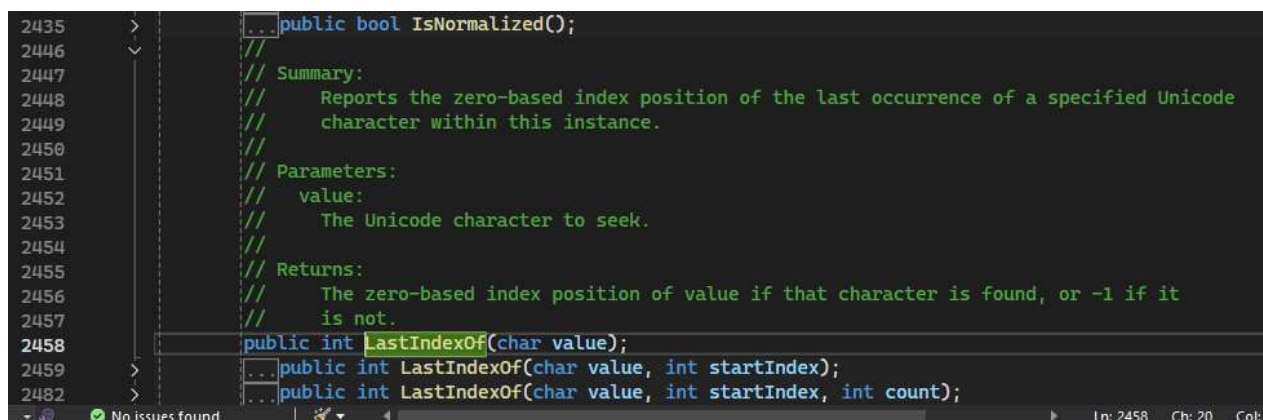
GitHub Examples and Documentation (Alt+O)
CS1501: No overload for method 'LastIndexOf' takes 0 arguments
Show potential fixes (Alt+Enter or Ctrl+.)

Прозорецът с подробното описание се отваря вдясно:

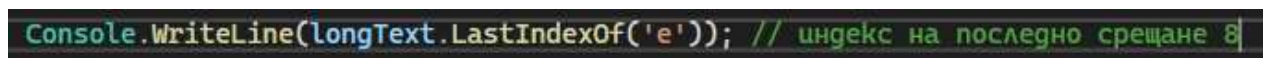


```
public int IndexOf(char value, StringComparison comparisonType);
public int IndexOf(char value);
public int IndexOfAny(char[] anyOf, int startIndex, int count);
public int IndexOfAny(char[] anyOf, int startIndex);
public int IndexOfAny(char[] anyOf);
public String Insert(int startIndex, String value);
public bool IsNormalized(NormalizationForm normalizationForm);
public bool IsNormalized();
public int LastIndexOf(char value);
public int LastIndexOf(char value, int startIndex);
public int LastIndexOf(char value, int startIndex, int count);
public int LastIndexOf(String value);
public int LastIndexOf(String value, int startIndex);
public int LastIndexOf(String value, int startIndex, int count);
public int LastIndexOf(String value, int count, StringComparison comparisonType);
public int LastIndexOf(String value, int startIndex, StringComparison comparisonType);
public int LastIndexOf(String value, StringComparison comparisonType);
public int LastIndexOfAny(char[] anyOf);
public int LastIndexOfAny(char[] anyOf, int startIndex, int count);
public int LastIndexOfAny(char[] anyOf, int startIndex);
public String Normalize();
```

Можем да разлистим и прочетем подробно summary над метода. Познато? **///** XML над class – персонални подсказки

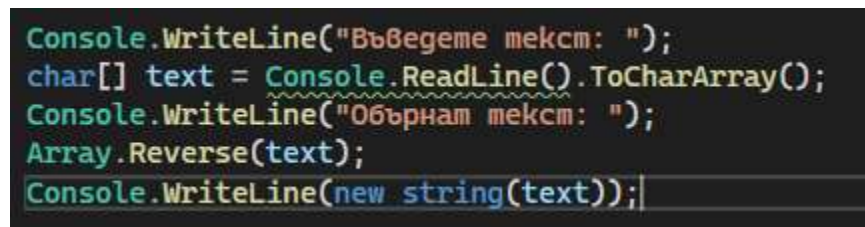


```
public bool IsNormalized();
// Summary:
// Reports the zero-based index position of the last occurrence of a specified Unicode
// character within this instance.
// Parameters:
// value:
// The Unicode character to seek.
// Returns:
// The zero-based index position of value if that character is found, or -1 if it
// is not.
public int LastIndexOf(char value);
public int LastIndexOf(char value, int startIndex);
public int LastIndexOf(char value, int startIndex, int count);
```



```
Console.WriteLine(longText.LastIndexOf('e')); // индекс на последно срещане 8
```

.Reverse() – обръщане на текст **.ToArray()** – добавяне на символи в масив



```
Console.WriteLine("Въведете текст: ");
char[] text = Console.ReadLine().ToArray();
Console.WriteLine("Обърнат текст: ");
Array.Reverse(text);
Console.WriteLine(new string(text));
```

```
Console.WriteLine("Въведете текст: ");
char[] text = Console.ReadLine().ToArray();
Console.WriteLine("Обърнат текст: ");
Array.Reverse(text);
Console.WriteLine(new string(text));
```

Последно за string – да припомним как сме обединявали функционалности, например за автоматично подчертаване на заглавие:

```
string title = "String Builder";  
Console.WriteLine($"=== {title.ToUpper()} ===");  
Console.WriteLine(new string('-', title.Length + 8));
```

По-точно обяснено, клас **String** предоставя вградени функционалности – той е библиотека, а **string** е псевдоним (alias).

String се състои от по-прост тип данни char и затова може да създава нови обекти чрез new.

```
string title = "String Builder";  
Console.WriteLine($"=== {title.ToUpper()} ===");  
Console.WriteLine(new string('-', title.Length + 8));
```

StringBuilder

Урок от SoftUni: <https://introprogramming.info/intro-csharp-book/read-online/glava13-simvolni-nizove/>

Понеже string е **{ get; }** – read-only и не може да бъде презаписван, всеки път, когато „променим“ низ, всъщност се създава НОВ низ в паметта. Старият низ остава в паметта, докато Garbage Collector го почисти. Това е много бавно при големи обеми от данни.

The StringBuilder Class

- **StringBuilder(int capacity)** constructor allocates in advance buffer of size capacity
- **Capacity** holds the currently allocated space (in characters)
- **Length** holds the length of the string in the buffer
- **this[int index]** (indexer) access the char at given position

Capacity

used buffer (Length) unused buffer

Класове:

String - използва се за фиксирани текстове или малки промени

StringBuilder - използва се за динамично конструиране на големи текстове. Той пази символите в масив, който може да расте динамично (т.к. има буфер от допълнителна свободна памет като резерв. Позволява директни промени без да създава нови обекти и е много по-бърз при множество операции.

Като асоциации за гъвкавост, String е подобен на масив[] – с ограничения, докато StringBuilder наподобява повече списък<> - с повече вградени възможности.

Ако имате повече от 3 конкатенации в цикъл, използвайте StringBuilder!

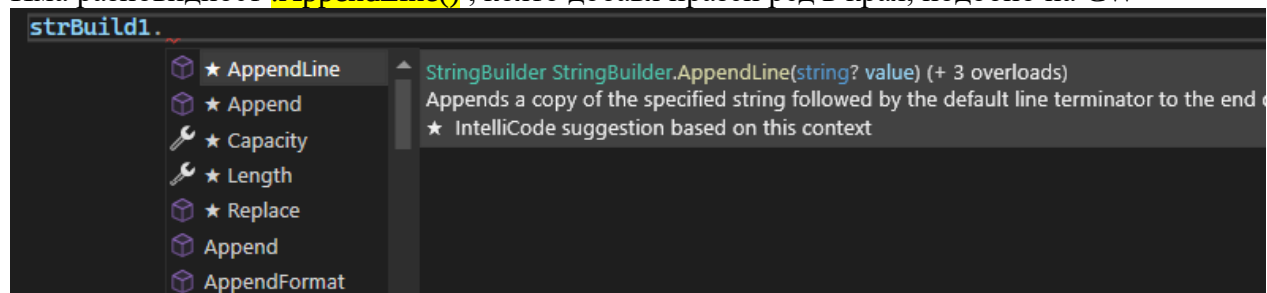
Създаване на нов обект от КЛАС StringBuilder чрез **new**:

```
StringBuilder strBuild1 = new StringBuilder(); // Нов, празен
StringBuilder strBuild2 = new StringBuilder("Нашият първи дълъг текст!");
StringBuilder strBuild3 = new StringBuilder(100); // първоначален капацитет
```

```
StringBuilder strBuild1 = new StringBuilder(); // Нов, празен
StringBuilder strBuild2 = new StringBuilder("Нашият първи дълъг текст!");
StringBuilder strBuild3 = new StringBuilder(100); // първоначален капацитет
```

Добавяне на текст чрез **.Append()** - // Апендикс = апандисит = придатък

Има разновидност **.AppendLine()**, която добавя празен ред в края, подобно на CW



Пример за замяна, свързване, верижно подреждане едно под друго за по-добра четимост (**chaining**), отпечатване:

```
strBuild1.AppendLine("Това е първият текст, последван от празен ред.");
strBuild2.Replace("първи", "втори")
    .AppendLine(" Това е чудесно!")
    .Append("Можем да добавим булева променлива: ")
    .Append(true)
    .Append(" А също и число: ")
    .Append(42);

Console.WriteLine(strBuild1);
Console.WriteLine(strBuild2);
```

```
strBuild1.AppendLine("Това е първият текст, последван от празен ред.");
```

```

strBuild2.Replace("първи", "втори")
    .AppendLine(" Това е чудесно!")
    .Append("Можем да добавим булева променлива: ")
    .Append(true)
    .Append(" А също и число: ")
    .Append(42);

Console.WriteLine(strBuild1);
Console.WriteLine(strBuild2);

```

Не се хвърля грешка при отпечатване на тип `StringBuilder` директно с `CW`, но това не е добра практика! Желателно е да използваме `.ToString` на последен етап, непосредствено преди отпечатването!

Резултат

```

=== STRING BUILDER ===
=====
Това е първият текст, последван от празен ред.

Нашият втори дълъг текст! Това е чудесно!
Можем да добавим булева променлива: True А също и число: 42

```

`.Insert()`
`.Remove()`
`.Clear()`

```

strBuild1.Insert(14, " интересен");
Console.WriteLine(strBuild1);

strBuild1.Remove(22, 27);    // от index 22 "интерес" премахва 27 символа до "ен рег"
Console.WriteLine(strBuild1);

strBuild2.Clear();
Console.WriteLine("Дължина: " + strBuild2.Length); // 0

```

```

strBuild1.Insert(14, " интересен");
Console.WriteLine(strBuild1);

strBuild1.Remove(22, 27);    // от index 22 "интерес" премахва 27 символа до "ен
ред"
Console.WriteLine(strBuild1);

strBuild2.Clear();
Console.WriteLine("Дължина: " + strBuild2.Length); // 0

```

```
Това е първият интересен текст, последван от празен ред.  
Това е първият интересен ред.  
Дължина: 0
```

При `StringBuilder` може да достъпваме и променяме елемент директно по `index[]` – а при `string` НЕ!

```
strBuild3.AppendLine("* Достъп до символи!");  
Console.WriteLine(strBuild3);  
Console.WriteLine(strBuild3[0] + " Невъзможно със string!");  
strBuild3[0] = '#';  
Console.WriteLine(strBuild3 + " Промяната е осъществена!");  
  
Console.WriteLine("Капацитет на първи: " + strBuild1.Capacity);  
Console.WriteLine("Капацитет на втори: " + strBuild2.Capacity);  
Console.WriteLine("Капацитет на трети: " + strBuild3.Capacity);
```

```
strBuild3.AppendLine("* Достъп до символи!");  
Console.WriteLine(strBuild3);  
Console.WriteLine(strBuild3[0] + " Невъзможно със string!");  
strBuild3[0] = '#';  
Console.WriteLine(strBuild3 + " Промяната е осъществена!");  
  
Console.WriteLine("Капацитет на първи: " + strBuild1.Capacity);  
Console.WriteLine("Капацитет на втори: " + strBuild2.Capacity);  
Console.WriteLine("Капацитет на трети: " + strBuild3.Capacity);
```

`.AppendFormat()` – за форматиране на сложни низове

```
strBuild2.AppendFormat("Дата: {0:dd.MM.yyyy}, Температура: {1:F1}°C",  
    DateTime.Now, -1.5);  
  
Console.WriteLine(strBuild2);  
Console.WriteLine("Капацитет на втори след промяна: " + strBuild2.Capacity);  
  
string finalString = strBuild2.ToString();  
Console.WriteLine(finalString);
```

`.ToString()` – допустимо е конвертиране в обикновен низ **чак на финала**, преди отпечатване. След него се губят много от вградените възможности на `StringBuilder`, които липсват в обикновения `string` като `.Capacity`.

Сравнение между `String` и `StringBuilder`

Задача за таймер, включвайки клас `StopWatch` от `System.Diagnostics`:

```
namespace _02_timer
```

```

{
    using System;
    using System.Text;
    using System.Diagnostics;

    internal class Program
    {
        static void Main()
        {
            int count = 50000;

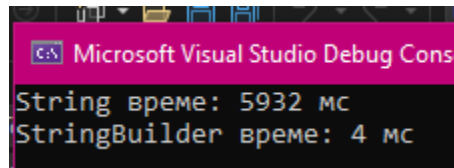
            // МЕТОД 1: Със string (бавно)
            Stopwatch timer = Stopwatch.StartNew();
            string result1 = "";
            for (int i = 0; i < count; i++)
            {
                result1 += i.ToString() + " ";
            }
            timer.Stop();
            Console.WriteLine($"String време: {timer.ElapsedMilliseconds} мс");

            // МЕТОД 2: Със StringBuilder (бързо)
            timer.Restart();
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < count; i++)
            {
                sb.Append(i).Append(" ");
            }
            string result2 = sb.ToString();
            timer.Stop();
            Console.WriteLine($"StringBuilder време: {timer.ElapsedMilliseconds}
мс");

            // Разликата може да е ДЕСЕТКИ пъти!
        }
    }
}

```

Изход:



```

Microsoft Visual Studio Debug Cons
String време: 5932 мс
StringBuilder време: 4 мс

```

Извод: Използваме StringBuilder когато:

- имаме много конкатенации в цикъл (над 10-20)
- Конструираме дълги текстове (HTML, SQL заявки, XML, JSON)
- Правим чести модификации на текста (вмъкване, замяна, премахване)
- Работим с текстове с неизвестна дължина

Цялият код:

```
using System.Text;
```

```

string shortText = "Сняг";
string longText = "Все още е зима";
string shortTextSmallCase = "сняг";

Console.WriteLine(string.Compare(shortText, longText)); // 1
Console.WriteLine(string.Compare(shortText, shortTextSmallCase, false)); // 1
Console.WriteLine(string.Compare(shortText, shortTextSmallCase, true)); // 0

Console.WriteLine(string.Concat(longText, " и вали ", shortText.ToLower()));

Console.WriteLine(longText.LastIndexOf('e')); // индекс на последно срещане 8

Console.WriteLine("Въведете текст: ");
char[] text = Console.ReadLine().ToCharArray();
Console.WriteLine("Обърнат текст: ");
Array.Reverse(text);
Console.WriteLine(new string(text));
Console.WriteLine();

string title = "String Builder";
Console.WriteLine($"=== {title.ToUpper()} ===");
Console.WriteLine(new string('=', title.Length + 8));

StringBuilder strBuild1 = new StringBuilder(); // Нов, празен
StringBuilder strBuild2 = new StringBuilder("Нашият първи дълъг текст!");
StringBuilder strBuild3 = new StringBuilder(100); // първоначален капацитет

strBuild1.AppendLine("Това е първият текст, последван от празен ред.");
strBuild2.Replace("първи", "втори")
    .AppendLine(" Това е чудесно!")
    .Append("Можем да добавим булева променлива: ")
    .Append(true)
    .Append(" А също и число: ")
    .Append(42);

Console.WriteLine(strBuild1);
Console.WriteLine(strBuild2);

strBuild1.Insert(14, " интересен");
Console.WriteLine(strBuild1);

strBuild1.Remove(22, 27); // от index 22 "интерес" премахва 27 символа до "ен
ред"
Console.WriteLine(strBuild1);

strBuild2.Clear();
Console.WriteLine("Дължина: " + strBuild2.Length); // 0

strBuild3.AppendLine("* Достъп до символи!");
Console.WriteLine(strBuild3);
Console.WriteLine(strBuild3[0] + " Невъзможно със string!");
strBuild3[0] = '#';
Console.WriteLine(strBuild3 + " Промяната е осъществена!");

Console.WriteLine("Капацитет на първи: " + strBuild1.Capacity);
Console.WriteLine("Капацитет на втори: " + strBuild2.Capacity);
Console.WriteLine("Капацитет на трети: " + strBuild3.Capacity);

```

```
strBuild2.AppendFormat("Дата: {0:dd.MM.yyyy}, Температура: {1:F1}°C",  
    DateTime.Now, -1.5);  
  
Console.WriteLine(strBuild2);  
Console.WriteLine("Капацитет на втори след промяна: " + strBuild2.Capacity);  
  
string finalString = strBuild2.ToString();  
Console.WriteLine(finalString);
```